

‘‘‘

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

CSE 4802: Software Maintenance Lab

Lab 3
Transformation Tools

Antara Arifa Mullick

Student ID: 210042132

Program: B.Sc. in SWE
Department of CSE

Task Author: Lutfun Nahar Lota
CSE, IUT

‘‘‘

1 Work Done

In this lab, the transformation tools shown in the lab manual were applied to my own working GitHub project named *Heart Disease Prediction*. The project is a Python-based machine learning project that predicts whether a person has heart disease or not using a Logistic Regression model.

The original project was in Jupyter Notebook format. For this lab, the same project workflow was converted into a Python script named `lab3_heart_disease_tools.py`. This was done because the tools used in this lab, especially VizTracer and SnakeViz, are easier to run from the terminal using a Python script.

The tools used in this lab were:

- Loguru
- PySnooper
- VizTracer
- SnakeViz

The main workflow of the selected project was:

1. Load the heart disease dataset.
2. Separate the feature columns and target column.
3. Split the dataset into training and testing sets.
4. Train a Logistic Regression model.
5. Evaluate the model using training and testing accuracy.
6. Predict heart disease for one sample input.
7. Apply transformation tools to observe logs, traces, execution flow, and profiling output.

2 Selected Working Project

The selected project was *Heart Disease Prediction*. This project was suitable for the lab because it is Python-based and has a clear execution flow. The project uses a dataset

named `heart_disease_data.csv`. The dataset contains 303 rows and 14 columns. The target column represents whether a person has heart disease or not.

The project was divided into separate functions so that the tools could show meaningful output. The main functions were:

- `load_dataset()` for loading the CSV dataset.
- `prepare_data()` for separating features and target values.
- `train_model()` for training the Logistic Regression model.
- `evaluate_model()` for calculating training and testing accuracy.
- `predict_sample()` for predicting heart disease for one sample input.
- `main()` for controlling the full execution flow.

This structure made the project easier to maintain because each function had a clear responsibility. It also made the output of VizTracer and SnakeViz more meaningful because the execution flow could be seen through separate function calls.

3 Tools and Environment

Table 1: Tools used in the lab

Tool	**Purpose**
Python	Programming language used for the Heart Disease Prediction project.
NumPy	Used for numerical data handling and sample input conversion.
Pandas	Used for reading and processing the CSV dataset.
scikit-learn	Used for train-test split, Logistic Regression model, and accuracy calculation.
Loguru	Used for structured logging of project execution.
PySnooper	Used for line-by-line tracing of the prediction function.
VizTracer	Used for visualizing execution flow as an HTML timeline.
cProfile	Used for generating profiling information.
SnakeViz	Used for visualizing cProfile output in the browser.

The required packages were installed using the following command:

Listing 1: Installing required packages

```
pip install numpy pandas scikit-learn loguru pysnooper viztracer
   snakeviz
```

4 Project Preparation

A new Python file named `lab3_heart_disease_tools.py` was created inside the project folder. The purpose of this file was to apply the lab tools to the existing Heart Disease Prediction project.

The project was first run normally using:

Listing 2: Running the project normally

```
python lab3_heart_disease_tools.py
```

The project executed successfully and produced the following result:

Listing 3: Normal project output

```
Accuracy on Training data: 0.8553719008264463
Accuracy on Test data: 0.8032786885245902
Prediction: [0]
The Person does not have a Heart Disease
```

This confirmed that the project was running correctly before analyzing the output of the transformation tools.

5 Tool 1: Loguru

5.1 Why Loguru Was Used

Loguru was used to record the execution flow of the project. Instead of only printing output in the terminal, Loguru allowed important events to be saved in a log file. This is useful because logs provide a permanent record of what happened during execution.

In this project, Loguru was used to log:

- Project start and completion.
- Dataset loading.
- Dataset shape and column names.
- Missing value count.
- Target value distribution.
- Training and testing data shape.

- Model training completion.
- Training and testing accuracy.
- Final prediction result.

Some examples of Loguru statements used in the project are:

Listing 4: Loguru usage in the project

```
logger.info("Loading dataset")
logger.success("Dataset loaded successfully")
logger.debug(f"Dataset shape: {heart_data.shape}")
logger.debug(f"Missing values in dataset: {heart_data.isnull().sum()
    .sum()}")
logger.info("Training Logistic Regression model")
logger.success("Model training completed")
logger.success(f"Accuracy on test data: {test_data_accuracy}")
logger.info(f"Final prediction result: {result}")
```

5.2 Output

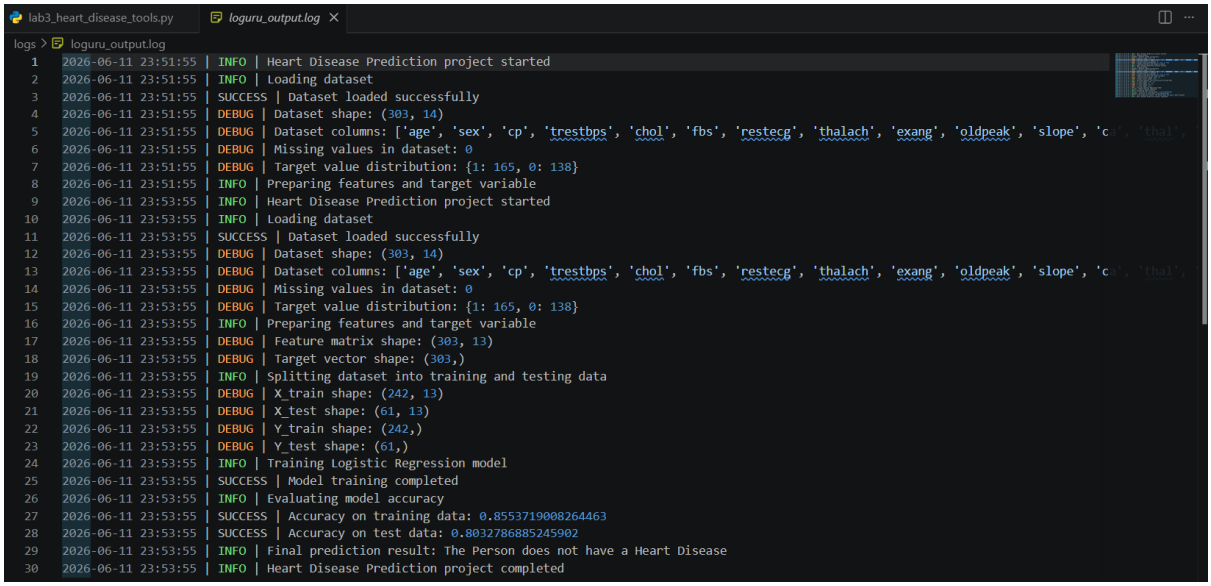


Figure 1: Loguru output showing project execution logs.

5.3 Insight from the Output

The Loguru output showed that the dataset was loaded successfully. It also showed that the dataset shape was `(303, 14)`, there were no missing values, and the target

distribution was `1: 165, 0: 138`.

The log file also showed that the data was split into training and testing sets. The training accuracy was 0.8553719008264463 and the testing accuracy was 0.8032786885245902. Finally, the log showed that the final prediction result was: *The Person does not have a Heart Disease.*

5.4 Maintenance Purpose

Loguru is useful in maintenance because it helps developers track what happened during execution. If the project gives a wrong output in the future, a maintainer can check the log file to see whether the dataset was loaded properly, whether the data shape changed, whether missing values appeared, or whether the model accuracy changed.

6 Tool 2: PySnooper

6.1 Why PySnooper Was Used

PySnooper was used to trace a specific function line by line. In this project, it was applied to the `predict_sample()` function because this function is responsible for taking one sample patient's data, converting it into the required format, passing it to the model, and returning the prediction.

Tracing the whole project would generate too much output. Therefore, only the prediction function was traced.

6.2 Output

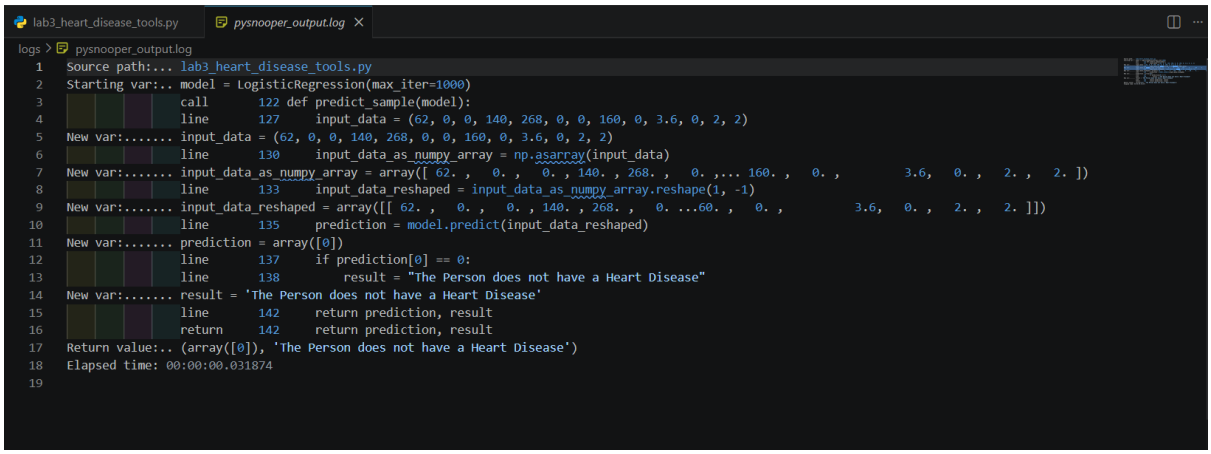


Figure 2: PySnooper output showing line-by-line execution of the prediction function.

6.3 Insight from the Output

The PySnooper output showed the execution of the `predict_sample()` function line by line. It showed the input tuple, the NumPy array conversion, the reshaped input array, the prediction value, and the final result message.

The prediction value was `array([0])`. Since the predicted value was 0, the program selected the message: *The Person does not have a Heart Disease*.

6.4 Maintenance Purpose

PySnooper is useful in maintenance when a particular function gives an unexpected result. In this project, if the prediction result seems wrong, PySnooper can help check whether the input data was converted correctly, whether the reshaping was correct, and whether the correct condition was executed after prediction.

7 Tool 3: VizTracer

7.1 Why VizTracer Was Used

VizTracer was used to visualize the runtime execution flow of the project. It generated an HTML timeline showing the execution behavior of the Python program.

7.2 Command Used

The following command was used:

Listing 5: VizTracer command

```
python -m viztracer -o outputs/viztracer_result.html
   lab3_heart_disease_tools.py
```

This command generated the output file:

Listing 6: VizTracer output file

```
outputs/viztracer_result.html
```

7.3 Output



Figure 3: VizTracer output showing execution timeline of the project.

7.4 Insight from the Output

The VizTracer output showed the main process and main thread of the project. The colored timeline blocks represented runtime activities and function calls. This helped visualize the execution sequence of the project.

From this output, the project flow can be understood as dataset loading, data preparation, model training, model evaluation, and sample prediction. During execution, VizTracer also showed a message that the circular buffer was full. This happened because many internal Python and machine learning library calls were also traced.

7.5 Maintenance Purpose

VizTracer is useful in maintenance because it gives a visual representation of how the program runs. A maintainer can use it to understand runtime flow without reading the entire source code line by line. It is especially useful when the program has many function calls or unexpected execution behavior.

8 Tool 4: SnakeViz

8.1 Why SnakeViz Was Used

SnakeViz was used to visualize the profiling output of the project. Profiling helps identify how long functions take to execute and where time is spent during program execution.

SnakeViz requires a `.profile` file generated by cProfile. Therefore, the project was first profiled using cProfile and then visualized using SnakeViz.

8.2 Commands Used

First, the profile file was generated:

Listing 7: Generating cProfile output

```
python -m cProfile -o outputs/profile_output.profile
   lab3_heart_disease_tools.py
```

Then, the profile output was opened using SnakeViz:

Listing 8: Opening cProfile output in SnakeViz

```
python -m snakeviz outputs/profile_output.profile
```

8.3 Output

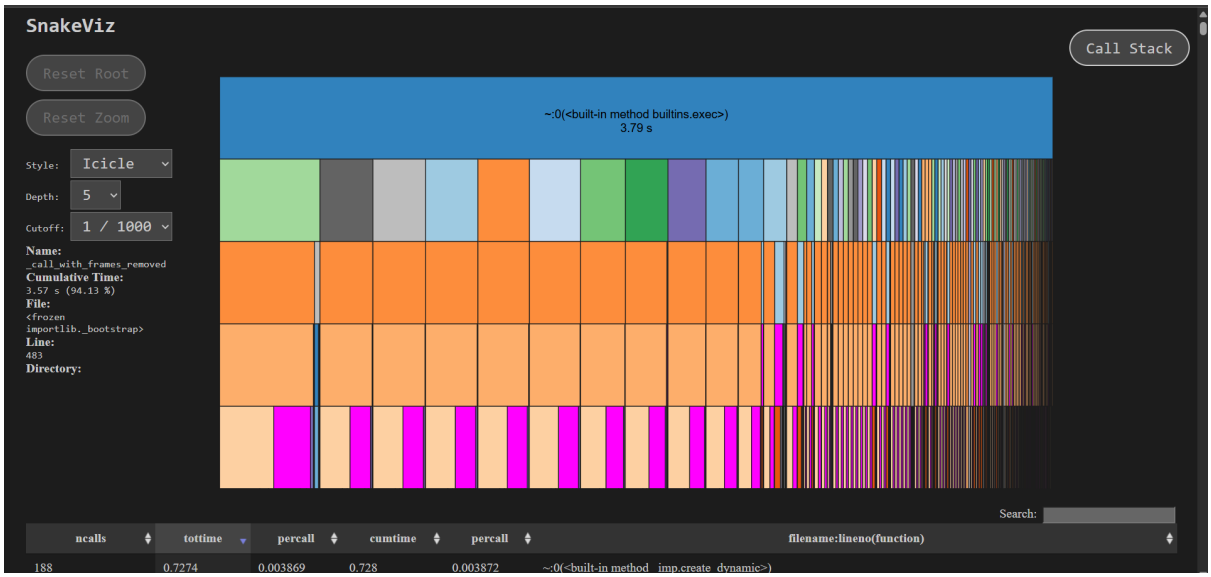


Figure 4: SnakeViz output showing profiling result as an icicle chart.

8.4 Insight from the Output

The SnakeViz output showed an icicle chart of the profiling result. The chart displayed how execution time was distributed across function calls. The total profiling time shown was approximately 3.79 seconds.

Since the full script was profiled, many internal Python and library functions also appeared in the output. The chart showed that execution time was not only spent in the project functions but also in internal imports and machine learning library operations.

8.5 Maintenance Purpose

SnakeViz is useful in maintenance when a project becomes slow. A maintainer can use SnakeViz to identify which functions or library calls consume more execution time. After identifying the expensive parts, the maintainer can optimize those parts of the code.

9 Summary of Outputs

Table 2: Summary of transformation tool outputs

Tool	Generated Output	Main Insight
Loguru	`logs/loguru_output.log`	Showed dataset loading, dataset shape, missing values, target distribution, model training, accuracy, and final prediction.
PySnooper	`logs/pysnooper_output.log`	Showed line-by-line execution and variable changes inside the prediction function.
VizTracer	`outputs/viztracer_result.html`	Showed execution flow and runtime timeline of the project.
SnakeViz	`outputs/profile_output.profile`	Showed profiling result and execution time distribution.

10 Model Result

The Logistic Regression model produced the following accuracy values:

Table 3: Accuracy result of the Heart Disease Prediction model

Metric	Value
Training Accuracy	0.8553719008264463
Testing Accuracy	0.8032786885245902

The final sample prediction was:

Listing 9: Final prediction output

```
Prediction: [0]
The Person does not have a Heart Disease
```

11 Alternative Tools

Table 4: Alternative tools

Tool Used in Lab	**Alternative Tools**
Loguru	Python logging module
PySnooper	VS Code Debugger, PyCharm Debugger
VizTracer	PyInstrument, Austin, Scalene
SnakeViz	PyCharm Profiler, cProfile terminal output, gprof2dot, Tuna

The default Python logging module can be used instead of Loguru, but Loguru is easier to configure and read for small and medium projects. For debugging, VS Code Debugger or PyCharm Debugger can be used instead of PySnooper. For execution profiling and visualization, PyInstrument, Scalene, gprof2dot, and Tuna can be used as alternatives to VizTracer and SnakeViz.

12 Preferred Tool

Among the four tools, I would prefer **Loguru** for regular software maintenance. The reason is that logging is useful throughout the lifetime of a software project. Once logging is added, it can continuously record important execution events such as dataset loading, model training, errors, accuracy values, and prediction results.

For debugging a specific function, I would prefer **PySnooper** because it clearly shows variable changes line by line. For understanding the overall execution flow visually, **VizTracer** is useful. For performance-related maintenance, **SnakeViz** is useful because it helps identify time-consuming parts of the program.

13 Discussion

This lab showed how transformation tools can help in software maintenance. Each tool gave a different view of the same project. Loguru showed what happened during execution. PySnooper showed how variable values changed inside a selected function. VizTracer showed how the program executed over time. SnakeViz showed where execution time was spent.

The Heart Disease Prediction project was suitable for this lab because it had a clear execution flow: dataset loading, data preparation, model training, evaluation, and prediction. Applying the tools to this project made the internal behavior of the program

easier to understand.

From a maintenance perspective, these tools are helpful because they reduce the difficulty of understanding existing code. Loguru provides a permanent record of execution, PySnooper helps debug a selected function, VizTracer provides a visual execution timeline, and SnakeViz supports performance analysis.

14 Conclusion

In this lab, four transformation and profiling tools were successfully applied to the Heart Disease Prediction project. Loguru generated structured logs, PySnooper traced the prediction function line by line, VizTracer generated an HTML execution timeline, and SnakeViz visualized the cProfile output as an icicle chart.

The outputs helped explain the behavior of the project from different perspectives. Loguru was useful for execution history, PySnooper was useful for debugging, VizTracer was useful for visualizing execution flow, and SnakeViz was useful for performance profiling. Among these tools, Loguru was the preferred tool for regular maintenance because it is simple, readable, and useful throughout the software development and maintenance process.